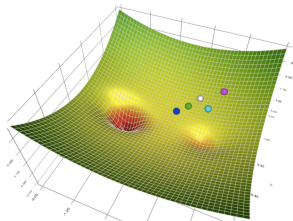


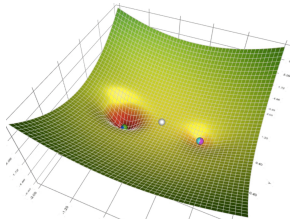
4 References

How to Update Weights?

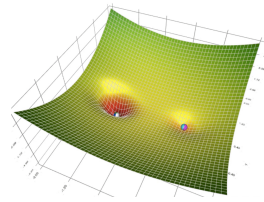
- Imagine training a large model like ChatGPT. It has billions of parameters that need to be adjusted.
- If we used **random search** to update these weights, it would take an astronomical number of trials to find good parameters.
- How to make training feasible at this massive scale?
- Image adapted from Medium: AdaGrad Optimization Algorithm



Visualizing parameter space with
random initial points.



Random search approach in
high-dimensional space.



Challenges in finding optimal weights using random search.

How to Update Weights?

Options for Updating Weights:

- **Random Search:** Tries values randomly—inefficient and impractical.
- **Gradient Descent:** Follows the slope of the loss function—efficient and guided.

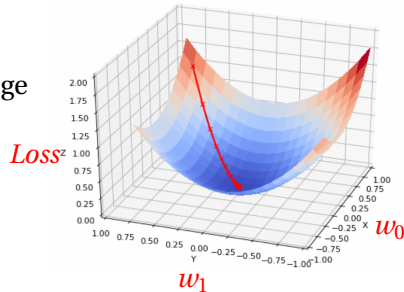
Why Gradient Descent?

- It updates weights by following the slope, reducing error with each step.
- Controlled, stepwise updates ensure we move closer to minimizing the loss effectively.

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

- Compute the gradient

- Compute the gradient of the loss function.
- Update the weights using the update rule.
- Repeat until convergence.
- Image adapted from Data Science Stack Exchange



2 Backpropagation

Vectorized Backpropagation

4 References

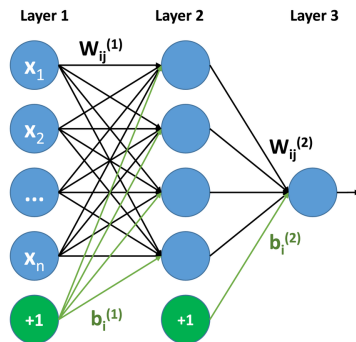
2 Backpropagation

Vectorized Backpropagation

4 References

Multi-Layer Neural Network

The diagram below illustrates a three-layer neural network

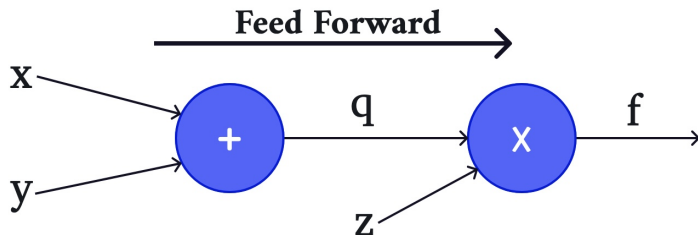


Neural Network

How should $\frac{\partial L}{\partial w}$ be calculated for each weight w in this neural network?

$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$



A Simple Example: Forward Pass

Function:

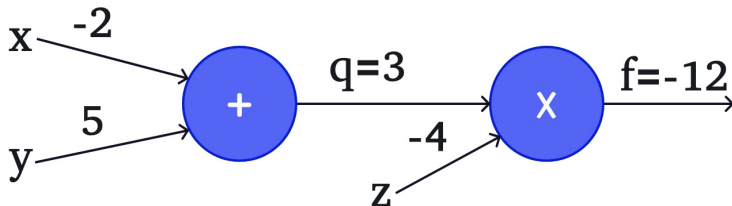
$$f(x, y, z) = (x + y)z$$

Example:

$$x = -2, \quad y = 5, \quad z = -4$$

Steps:

- $q = x + y = 3$
- $f = q \times z = -12$



1 1 0

- (c) (d) (e)

1

- 1 2 3

Chain Rule for Gradients

The **chain rule** helps us find the gradient of a function that is composed of other functions.

Example:

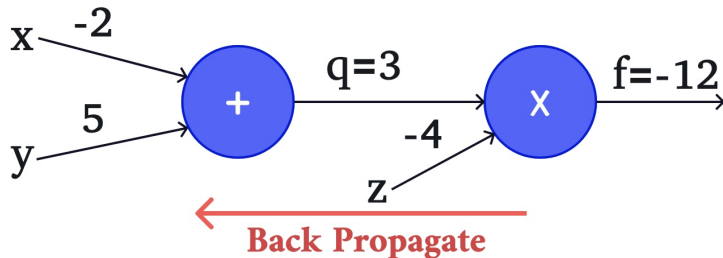
$$z = f(g(x))$$

- f is a function of $g(x)$
- $g(x)$ is a function of x

To find $\frac{\partial z}{\partial x}$, we use the chain rule:

$$\frac{\partial z}{\partial x} = \frac{\partial f}{\partial g} \times \frac{\partial g}{\partial x}$$

This means we multiply the gradient of the outer function by the gradient of the inner function.

$$f(x, y, z) = (x + y)z$$
$$x = -2, \quad y = 5, \quad z = -4$$


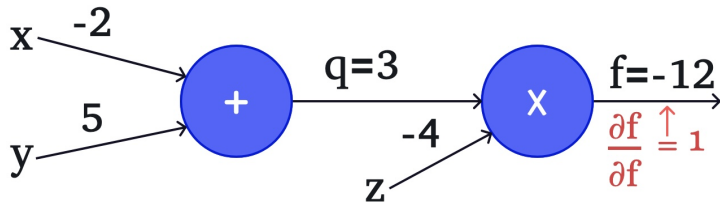
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100

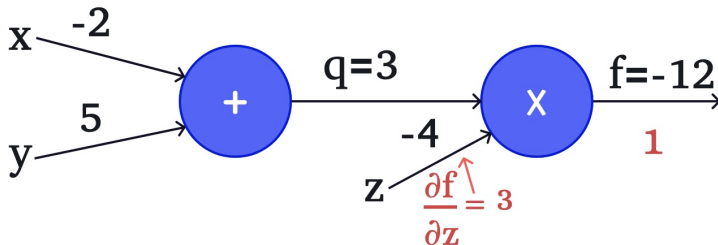
• • •

1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 2680, 26

100

100



$$f(x, y, z) = (x + y)z$$
$$x = -2, \quad y = 5, \quad z = -4$$
$$f = qz, \quad \frac{\partial f}{\partial z} = q = 3$$


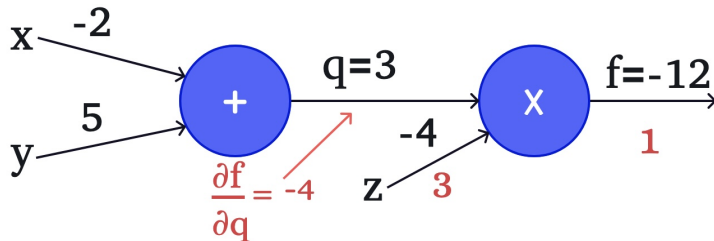
$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$f = qz, \quad \frac{\partial f}{\partial z} = q = 3$$

$$\frac{\partial f}{\partial q} = z = -4$$



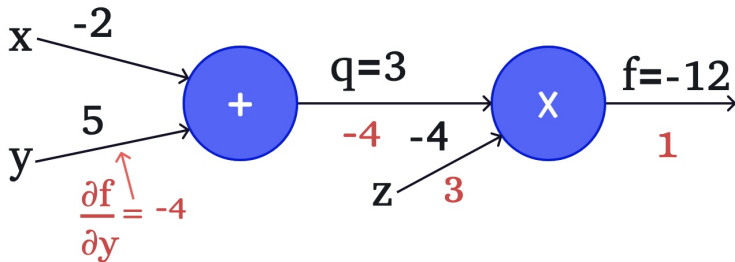
$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y} = -4 \cdot 1 = -4$$



$$f(x,y,z) = (x+y)z$$

$$f(x, y, z) = (x + y)z$$

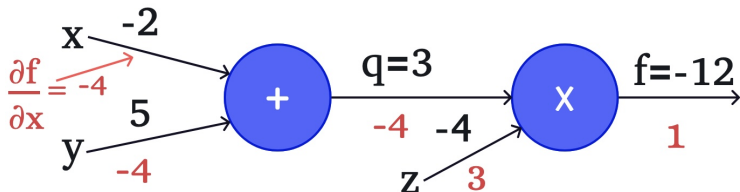
$$x = -2, \quad y = 5, \quad z = -4$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = -4 \cdot 1 = -4$$



$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

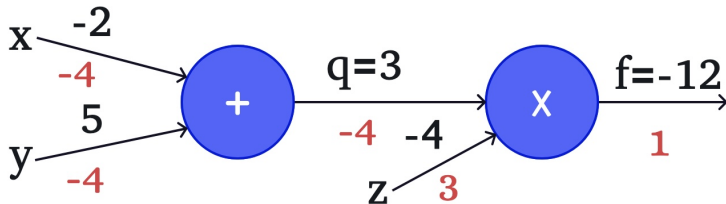
$$x = -2, \quad y = 5, \quad z = -4$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = -4 \cdot 1 = -4$$



Example: One Neuron

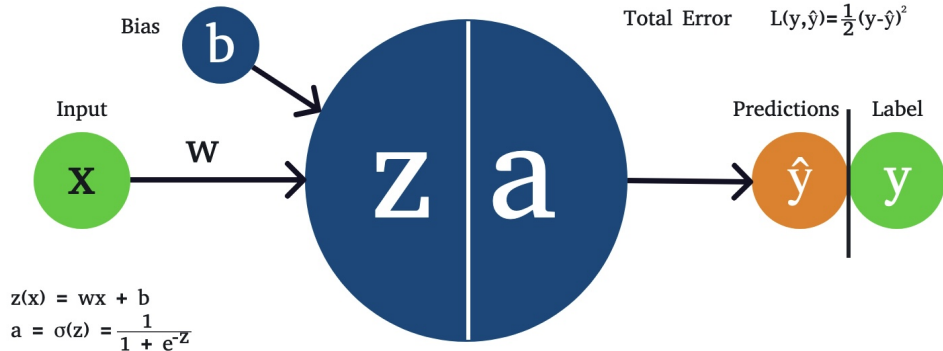
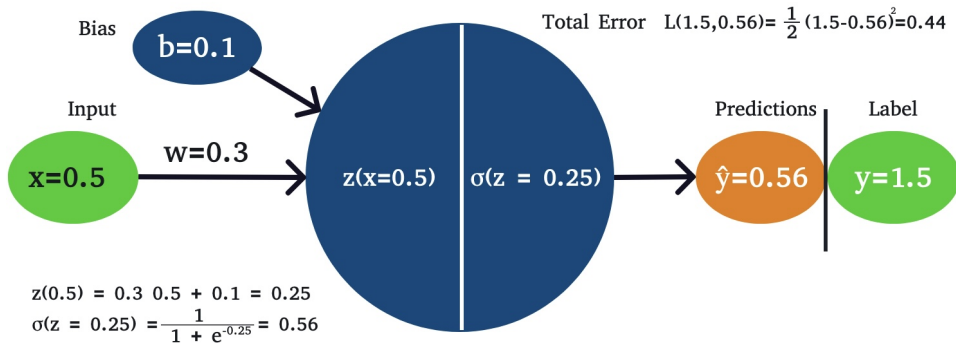
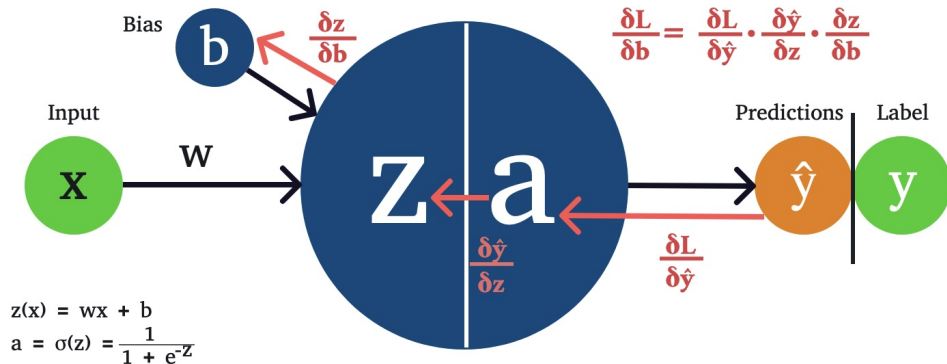


Image adapted from “Deep learning backpropagation” Web article

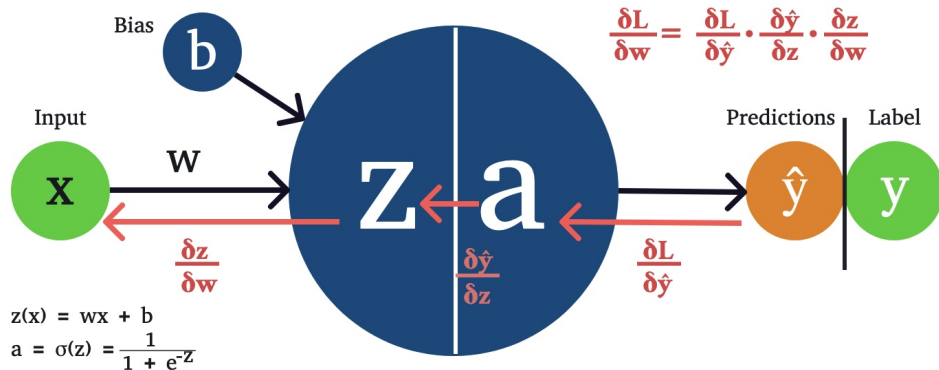
Forward Pass



Backward Pass



Backward Pass



Backward Pass



① Gradient Descent

② Backpropagation

Forward and Backward Passes

Vectorized Backpropagation

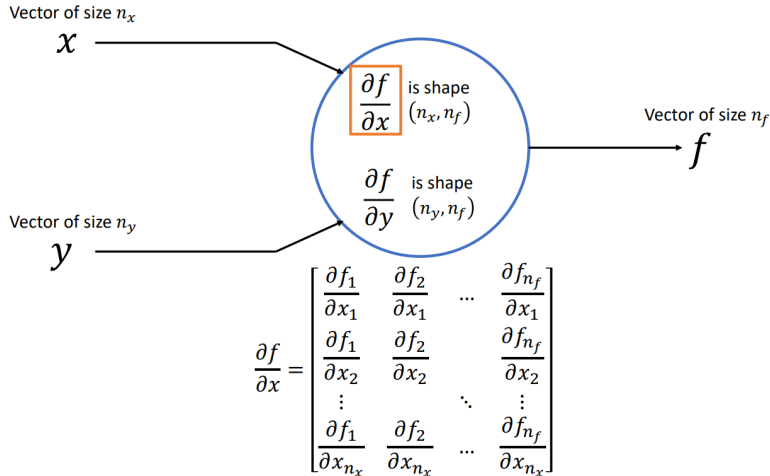
③ Foundations in Detail: Initialization, Loss, and Activation

④ References

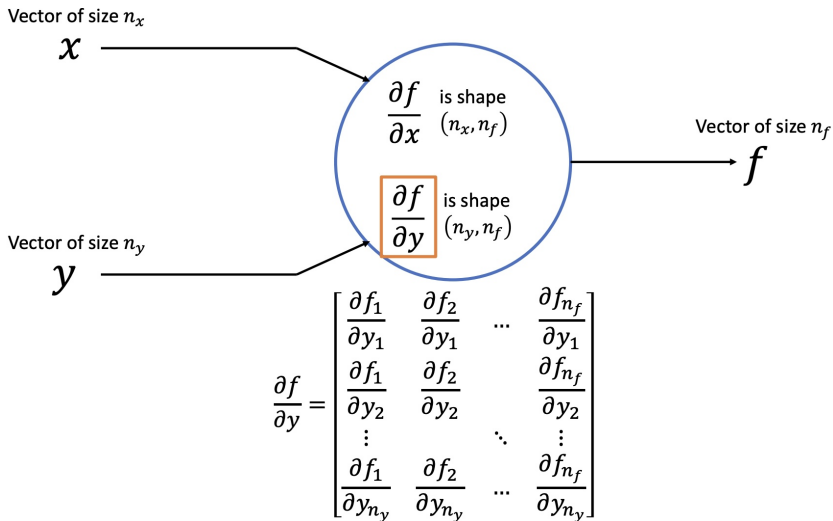
Derivative of a Vector by a Vector: leveraging matrix operations for faster computation.



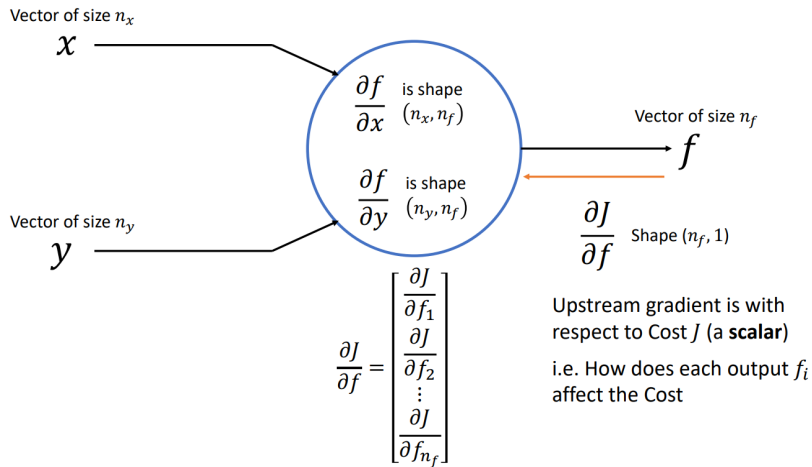
Local Derivatives are Jacobian Matrices



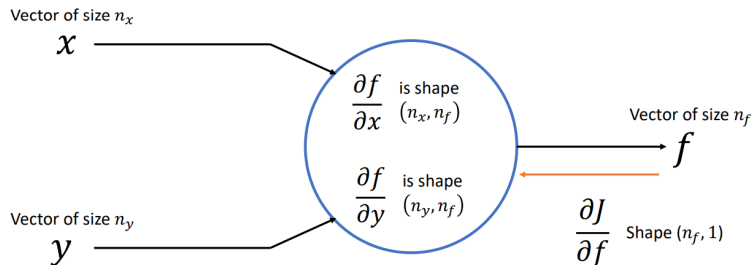
1. *Journal of the American Medical Association*, 1997; 277: 1039-1043.



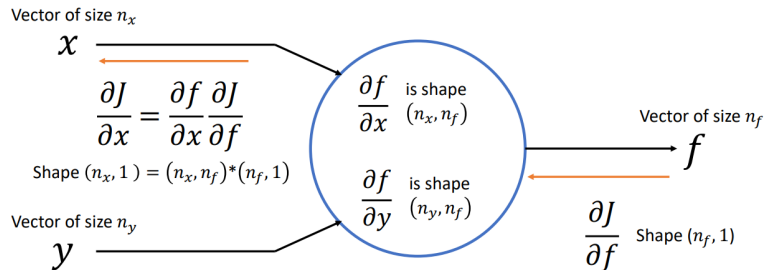
1. *Journal of the American Medical Association*, 1997; 277: 1001-1005.



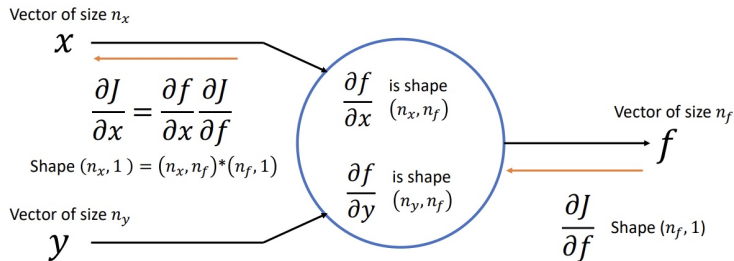
Vectorized Backpropagation



Apply chain rule like before!



Applying the chain rule involves matrix-vector multiplication



$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

Chain Rule – Matrix-Vector Multiply

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

$$\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$$

Chain Rule – Matrix-Vector Multiply

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

Jacobian

Shape $(n_x, 1) = (n_x, n_f) * (n_f, 1)$

Chain Rule – Matrix-Vector Multiply

The diagram illustrates the chain rule for backpropagation through a Jacobian matrix. It shows the relationship between the total derivative of the loss J with respect to an input x and the product of the Jacobian matrix and the upstream gradient.

The equation shown is:

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

Annotations in the diagram include:

- A blue arrow pointing from the text "Jacobian" to the Jacobian matrix.
- An orange arrow pointing from the text "Upstream Gradient" to the vector $\frac{\partial J}{\partial f}$.
- A blue arrow pointing from the text $\frac{\partial J}{\partial x}$ to the first term in the chain rule.
- An orange arrow pointing from the text $\frac{\partial f}{\partial x}$ to the second term in the chain rule.

The shape of the Jacobian matrix is given as:

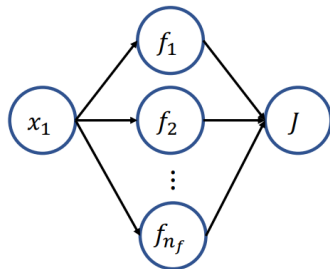
$$\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$$

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

$$\text{Shape}(n_x, 1) = (n_x, n_f)^* (n_f, 1)$$

$$\frac{\partial J}{\partial x_1} = \frac{\partial f_1}{\partial x_1} \frac{\partial J}{\partial f_1} + \frac{\partial f_2}{\partial x_1} \frac{\partial J}{\partial f_2} + \dots + \frac{\partial f_{n_f}}{\partial x_1} \frac{\partial J}{\partial f_{n_f}}$$

Chain Rule – Matrix-Vector Multiply



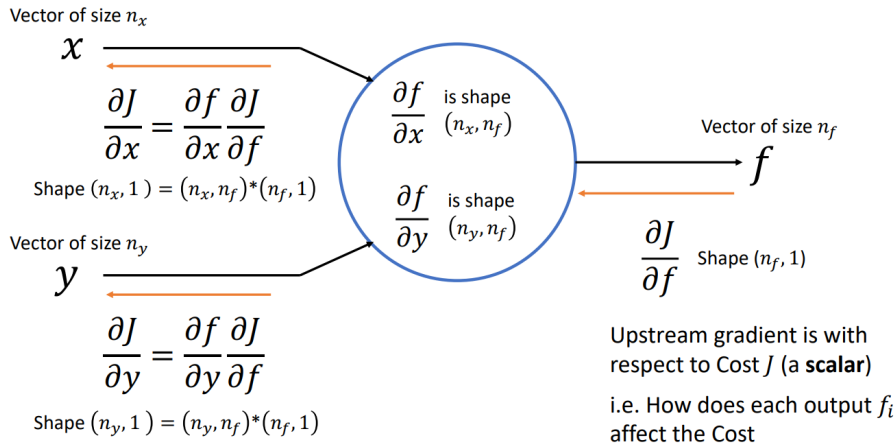
$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow$$

$$\begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

$$\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$$

$$\frac{\partial J}{\partial x_1} = \frac{\partial f_1}{\partial x_1} \frac{\partial J}{\partial f_1} + \frac{\partial f_2}{\partial x_1} \frac{\partial J}{\partial f_2} + \dots + \frac{\partial f_{n_f}}{\partial x_1} \frac{\partial J}{\partial f_{n_f}}$$

Downloaded from <http://ajphaphysocpharm.sagepub.com/> at 11:00 11 November 2014



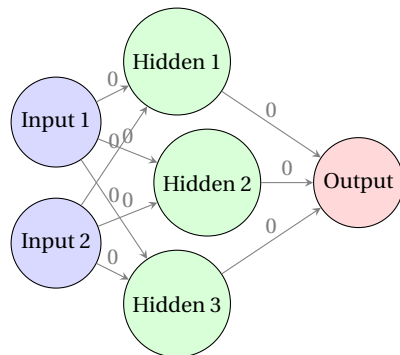
Chain Rule application is Matrix-Vector Multiply

Weight Initialization

Example: Imagine a network where all weights are initialized to zero.

Issue: If all weights are zero, each neuron in a layer will produce **identical** outputs. This symmetry prevents the network from learning **distinct features**, as every neuron updates identically.

Solution: To break this symmetry, weights need to be initialized with small random values, allowing neurons to learn unique features and avoid identical updates.



All weights initialized to zero

Xavier Initialization

Description: Xavier Initialization is designed to keep the variance of activations consistent across layers, ideal for **sigmoid** and **tanh** activations.

Objective: Prevents the shrinking or exploding of signal magnitudes during forward and backward propagation.

Condition:

$$\frac{1}{n_l} \text{Var}[w] = 1$$

where n_l is the number of neurons in layer l .

Initialization Scheme:

$$w \sim \mathcal{U}\left(-\sqrt{\frac{1}{n_l}}, \sqrt{\frac{1}{n_l}}\right)$$

This results in a uniform distribution within the range $-\sqrt{\frac{1}{n_l}}$ to $\sqrt{\frac{1}{n_l}}$, ensuring stable signal variance across layers.

Condition:

$$\frac{1}{2}n_l \text{Var}[w] = 1$$

Initialization Scheme:

$$w_l \sim \mathcal{N}\left(0, \frac{2}{n_l}\right)$$

This implies a zero-centered Gaussian distribution with a standard deviation of $\sqrt{\frac{2}{n_l}}$, where biases are initialized to 0.

1 Gradient Descent

2 Backpropagation

3 Foundations in Detail: Initialization, Loss, and Activation

Weight Initialization

Loss Functions

Activation Functions

4 References

Mean Squared Error (MSE)

Definition:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

Characteristics:

- Amplifies larger errors due to squaring, making it sensitive to outliers.

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

- **Example:** $[5.0, 4.0, 4.0]$

$$\text{MAE} = \frac{1}{3} (|5.0 - 4.2| + |4.0 - 3.8| + |4.9 - 5.1|)$$

$$= \frac{1}{3}(0.8 + 0.2 + 0.2) = \frac{1.2}{3} \approx 0.4$$

Binary Classification Loss – Binary Cross-Entropy

Binary Cross-Entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Example:

- Predicted probabilities: $\hat{y} = [0.7, 0.3, 0.9]$
- True labels: $y = [1, 0, 1]$

$$\begin{aligned}\mathcal{L}_{\text{BCE}} &= -\frac{1}{3} \left[1 \cdot \log(0.7) + (1-1) \cdot \log(1-0.7) \right. \\ &\quad \left. + 0 \cdot \log(0.3) + (1-0) \cdot \log(1-0.3) + 1 \cdot \log(0.9) + (1-1) \cdot \log(1-0.9) \right] \\ &= -\frac{1}{3} (\log(0.7) + \log(0.7) + \log(0.9)) \approx -\frac{1}{3} (-0.357 + -0.357 + -0.105) \approx 0.273\end{aligned}$$

Used to minimize the error between predicted probabilities and binary labels.

Categorical Cross-Entropy

Categorical Cross-Entropy Formula:

$$\mathcal{L}_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_c^{(i)} \log(\hat{y}_c^{(i)})$$

One-Hot Encoding

One-hot encoding represents categorical variables as binary vectors, with a 1 indicating the actual class and 0s elsewhere.

Example:

- Class 1: [1, 0, 0]
- Class 2: [0, 1, 0]
- Class 3: [0, 0, 1]

Example Calculation (3-Class)

Given:

- True labels (One-hot): Class 2, Class 1, Class 3
- Predicted probabilities:

$$\hat{y} = \underbrace{\begin{bmatrix} 0.1 & 0.7 & 0.2 \\ 0.6 & 0.3 & 0.1 \\ 0.1 & 0.6 & 0.3 \end{bmatrix}}_{\text{Classes}} \quad \text{Samples}$$

100

- ① True labels:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- ② Calculation:

$$\mathcal{L}_{CCE} = -\frac{1}{3} (\log(0.7) + \log(0.6) + \log(0.3))$$

- ③ Compute log terms:

$$\log(0.7) \approx -0.357, \quad \log(0.6) \approx -0.511, \quad \log(0.3) \approx -1.204$$

$$\mathcal{L}_{CCE} = \frac{1}{3} \times 2.072 \approx 0.691$$

Limitation of Linear Activation

Why Transform Outputs? Raw outputs need to be transformed into meaningful values, such as probabilities.

The Problem: Linear activation lacks non-linearity, restricting the model to simple linear relationships.



Neural Networks: Why is the Max Operator Important?

- **Before:** Linear score function:

$$f = Wx$$

- **Now:** 2-layer Neural Network:

$$f = W_2 \max(0, W_1 x)$$

- The function $\max(0, z)$ is called an activation function (in this case, ReLU).
- **Q: What if we try to build a neural network without an activation function?**

$$f = W_2 W_1 x$$

$$W_3 = W_2 W_1 \in \mathbb{R}^{C \times H}, \quad f = W_3 x$$

- **A:** We end up with a linear classifier again!

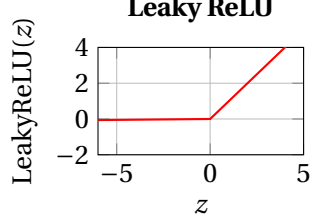
ReLU

Limitation:

- **Dead ReLU Problem:** Neurons can become inactive during training, outputting 0 for all inputs if they receive negative values consistently.

Question:

- Why does ReLU lead to faster training in deep networks?

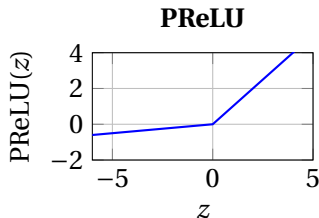


Variants of ReLU: PReLU (Parametric ReLU)

- Similar to Leaky ReLU, but the slope for negative inputs (α) is learned during training.

$$\text{PReLU}(z) = \max(\alpha z, z), \quad \alpha \text{ is learned}$$

- Provides more flexibility by adjusting the slope for negative inputs based on data.



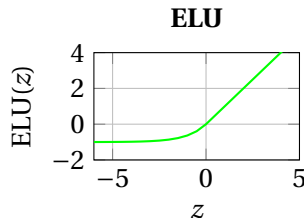
PReLU: Similar to Leaky ReLU, with a learnable slope.

Variants of ReLU: ELU (Exponential Linear Unit)

- Similar to ReLU for positive values but smoother for negative inputs.

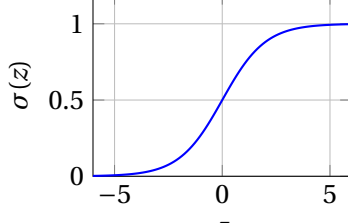
$$\text{ELU}(z) = \begin{cases} z, & \text{if } z > 0 \\ \alpha(e^z - 1), & \text{if } z \leq 0 \end{cases}, \quad \alpha = 1$$

- Provides faster convergence and reduces bias shift by smoothing negative values.



ELU: Smoother than ReLU for negative values.

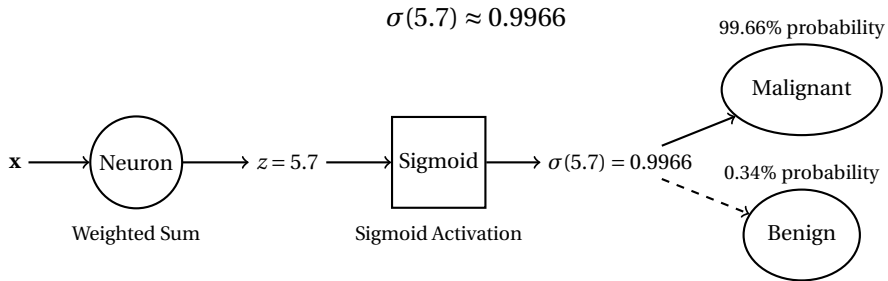
- | | | | | |
|--|--|--|--|--|
| | | | | |
|--|--|--|--|--|



Classification: Tumor Detection (Malignant vs. Benign)

Sigmoid Activation: Useful for binary classification!

- Example: For $z = 5.7$,



Sigmoid

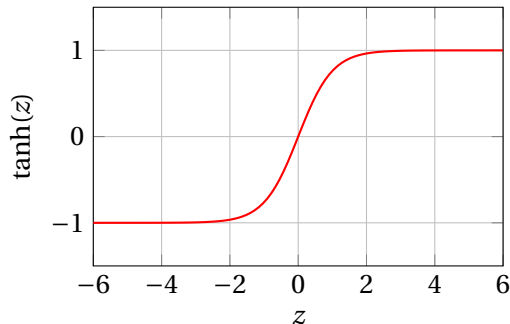
Limitations of Sigmoid:

- **Gradient Saturation:** When z is very large or very small, the gradient becomes nearly zero, causing slow learning (vanishing gradient problem).
- **Not Zero-Centered:** The output is not zero-centered, which can make optimization more difficult.

Question:

- Why does the vanishing gradient problem occur with Sigmoid during backpropagation? (To be discussed in more detail later)

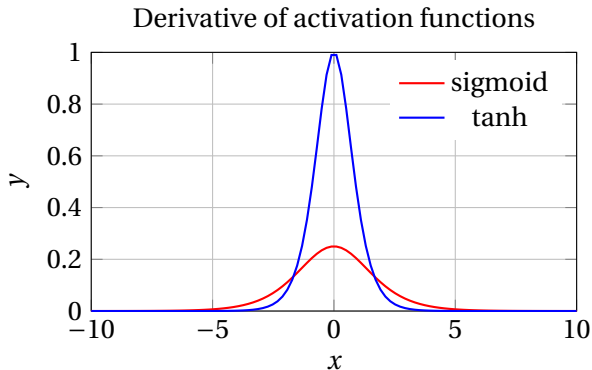
Tanh (Hyperbolic Tangent)



Question:

- How does Tanh help with faster convergence compared to Sigmoid?

1



- The derivative of the Tanh function has a much steeper slope at $x = 0$, meaning it provides a larger gradient for backpropagation compared to the Sigmoid function.

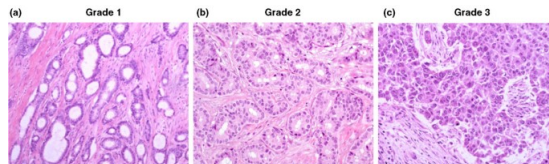
Problem: Multi-Class Tumor Classification

Scenario: We want to classify a tumor into one of three categories:

- **Class 0:** Benign
- **Class 1:** Malignant
- **Class 2:** Pre-cancerous

Goal: Given a set of tumor features, predict which class the tumor belongs to.

This is a **multi-class classification problem**, and we will use the **Softmax activation function** to assign probabilities to each class.



Microscopic images of tumor tissue, classified into grades representing different severity levels. Image adapted from Visual Analytics in Digital Computational Pathology

Example: Softmax Calculation

Consider a tumor with the following logits from a neural network:

- Logit for Benign (Class 0): $z_0 = 1.5$
- Logit for Malignant (Class 1): $z_1 = 0.8$
- Logit for Pre-Cancerous (Class 2): $z_2 = -0.5$

Step 1: Exponentiate the logits

$$e^{z_0} = e^{1.5} \approx 4.48, \quad e^{z_1} = e^{0.8} \approx 2.23, \quad e^{z_2} = e^{-0.5} \approx 0.61$$

Step 2: Compute the sum of exponentials

$$\text{Sum} = e^{z_0} + e^{z_1} + e^{z_2} = 4.48 + 2.23 + 0.61 = 7.32$$

Example: Softmax Probabilities

Step 3: Calculate Softmax probabilities for each class

$$P(\text{Benign}) = \frac{4.48}{7.32} \approx 0.612, \quad P(\text{Malignant}) = \frac{2.23}{7.32} \approx 0.305, \quad P(\text{Pre-Cancerous}) = \frac{0.61}{7.32} \approx 0.083$$

Step 4: Make a classification decision

- The highest probability is 0.612 for the **Benign** class.
- Therefore, the model predicts that the tumor is **Benign** (Class 0).

Conclusion: Softmax Activation for Classification

Key Points:

- Softmax is used in the output layer for **multi-class classification**.
- It converts logits into a **probability distribution** across classes.
- The class with the highest probability is selected as the prediction.

Main Idea: Softmax ensures all outputs sum to 1, making it ideal for choosing one class out of multiple options.

- [1] T. Networks, “Benign vs malignant tumors.” *Web article*, 2023.
<https://www.technologynetworks.com/cancer-research/articles/benign-vs-malignant-tumors-364765>.
- [2] B. Central, “Breast cancer research article.” *Journal article*, 2010.
<https://breast-cancer-research.biomedcentral.com/articles/10.1186/bcr2607>.
- [3] S. Raschka, “Gradient optimization image.” *Image*, 2020.
<https://sebastianraschka.com/images/faq/gradient-optimization/ball.png>.
- [4] Xnought, “Backpropagation explainer.” *Website*, 2022.
<https://xnought.github.io/backprop-explainer/>.
- [5] Datamapu, “Deep learning backpropagation.” *Web article*, 2021.
https://datamapu.com/posts/deep_learning/backpropagation/.

- https://www.researchgate.net/figure/Our-neural-network-settings-weight-matrix-W-and-bias-b-are-recalibrated-fig1_335468765

Any Questions?